

# EMMA — The complete guide

A self-hosted personal assistant · text, memory and tools

3 September 2026

# Contents

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>Introduction</b>                                               | <b>3</b>  |
| <b>Chapter 1 — Preparing the environment</b>                      | <b>4</b>  |
| 1.1 Which Ubuntu, and why                                         | 4         |
| 1.2 Checking your starting point                                  | 4         |
| 1.3 Updating the system                                           | 5         |
| 1.4 System packages                                               | 5         |
| 1.5 A dedicated system user                                       | 6         |
| 1.6 The installation directory and its permissions                | 6         |
| 1.7 The second disk, for backups                                  | 6         |
| 1.8 Firewall                                                      | 8         |
| 1.9 The credentials you need                                      | 8         |
| <b>Chapter 2 — Architecture</b>                                   | <b>9</b>  |
| 2.1 The overall picture                                           | 9         |
| 2.2 The path of a message                                         | 9         |
| 2.3 The adapter pattern: why core/ does not know what Telegram is | 10        |
| 2.4 The agentic router: the loop, and what it now has to call     | 10        |
| 2.5 Memory behind an interface                                    | 11        |
| 2.6 Resilience: three levels                                      | 12        |
| 2.7 Why FastAPI if there is nothing to expose                     | 12        |
| 2.8 The provider and the model                                    | 13        |
| <b>Chapter 3 — Implementation</b>                                 | <b>15</b> |
| 3.1 The map                                                       | 15        |
| 3.2 config.py — the configuration                                 | 15        |
| 3.3 core/memory.py — the memory                                   | 16        |
| 3.3bis core/tasks.py and tools/ — commissioning development       | 17        |
| 3.4 core/llm.py — the model client                                | 19        |
| 3.5 core/router.py — the orchestrator                             | 19        |
| 3.6 adapters/telegram.py — the channel                            | 20        |
| 3.7 main.py — startup                                             | 20        |
| 3.8 tests/ — the suite                                            | 20        |
| <b>Chapter 4 — Deployment</b>                                     | <b>22</b> |
| 4.1 The repository on GitHub                                      | 22        |
| 4.2 Fetching the code onto the server                             | 22        |
| 4.3 The virtual environment                                       | 22        |
| 4.4 The .env file                                                 | 23        |
| 4.5 The first start, by hand                                      | 23        |
| 4.6 The systemd service                                           | 24        |
| 4.7 The backup timer                                              | 25        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| 4.8 Summary: what is now on the machine . . . . .                 | 25        |
| 4.9 The development queue . . . . .                               | 26        |
| <b>Chapter 5 — Daily use</b>                                      | <b>30</b> |
| 5.1 Day to day . . . . .                                          | 30        |
| 5.2 What to expect . . . . .                                      | 30        |
| 5.3 Personalising her character . . . . .                         | 30        |
| 5.4 What it costs . . . . .                                       | 31        |
| 5.5 Known limits . . . . .                                        | 31        |
| 5.6 Commissioning development . . . . .                           | 31        |
| 5.7 Clearing the memory . . . . .                                 | 32        |
| <b>Chapter 6 — Maintenance</b>                                    | <b>33</b> |
| 6.1 Reading the logs . . . . .                                    | 33        |
| 6.2 The golden rule: back up first . . . . .                      | 34        |
| 6.3 Updating the code: development PC → GitHub → server . . . . . | 34        |
| 6.4 Updating the pinned dependencies . . . . .                    | 35        |
| 6.5 Backing up the configuration and the memory . . . . .         | 36        |
| 6.6 Restoring . . . . .                                           | 36        |
| 6.7 Common problems . . . . .                                     | 38        |
| 6.8 A maintenance calendar . . . . .                              | 41        |
| <b>Appendix A — Reference commands</b>                            | <b>42</b> |
| <b>Appendix B — The .env variables</b>                            | <b>43</b> |
| <b>Appendix C — Where to look when something does not add up</b>  | <b>44</b> |

# Introduction

This guide takes a freshly installed Ubuntu server to a working EMMA: a personal assistant you chat with from Telegram, running on hardware you own, talking to a language model through either the Anthropic API or the Groq one, as you prefer.

It is the complete reference manual. The repository's `README.md` is the short version for someone who only wants to try it; this guide is the one to follow to install it for real, understand it, and keep it running over time. Read it from beginning to end and you will not need to look anything up elsewhere.

**What EMMA does.** You write to your Telegram bot from your phone. A Python service on your server receives the message, adds the recent conversation and what it has been told to remember, asks the model for an answer and sends it back. The bot answers you and nobody else. The model is a configuration choice: Claude through the Anthropic API, or Groq, which has a free tier. The conversation is stored on disk and survives a restart. There are eleven tools — a clock, facts that do not expire, an inventory of her own tools, and the set that lets her be asked to build her own next feature. There is no voice yet; that is the next version, and it arrives as a new adapter rather than a rewrite.

**Who it is for.** Someone who can find their way around a Linux terminal but takes nothing for granted. Every command is written out in full and every choice is justified, because in six months you will want to know not only *what* you did but *why*.

## Conventions.

- Commands prefixed with `sudo` are to be run by a user with administrative privileges; the others by your ordinary user.
- Where `/opt/emma` appears, that is the installation directory: if you change it, change it everywhere (in this guide and in the files under `systemd/`, where the lines to touch are marked `# PATH:`).
- Where `<your-account>` appears, substitute your GitHub username.
- Blocks marked **Check** say what you must see to be sure the step worked. Do not skip them: this is how you avoid discovering a mistake three chapters later.

## The six chapters.

1. **Preparing the environment** — from a bare server to a system that is ready.
2. **Architecture** — how EMMA is built, and why.
3. **Implementation** — the code, file by file.
4. **Deployment** — configuration, the `systemd` service, the first start.
5. **Daily use** — using it from your phone, and the known limits.
6. **Maintenance** — logs, updates, backups, restores, common problems.

# Chapter 1 — Preparing the environment

## 1.1 Which Ubuntu, and why

**The choice: Ubuntu Server 24.04 LTS (Noble Numbat).**

The LTS releases currently supported are 24.04 (April 2024, standard support until April 2029) and 26.04, released in 2026 and therefore with the longest support window. For this project I choose 24.04 for three concrete reasons:

- **it is mature.** It has years of point releases behind it: the teething bugs in drivers, networking and the installer are fixed, and on a recycled old PC — which is exactly your case — hardware compatibility that has already been proven is worth more than any novelty.
- **third-party documentation is enormous.** When something does not work, searching for an error message finds answers written for 24.04. On a just-released LTS you often end up translating instructions written for the previous one.
- **it has Python 3.12**, which satisfies the project's 3.11+ requirement with room to spare.

26.04 LTS is an equally legitimate choice if you prefer the longer support: EMMA runs on it unmodified (Python 3.13, also compatible). Every command in this guide applies to both. What I do **not** recommend is an interim non-LTS release: nine months of support means a major upgrade almost every year on a machine whose only job is to stay switched on and work.

**Download the Server variant only**, not Desktop: no graphical environment means less RAM used, fewer packages to update and less attack surface. The image is at <https://ubuntu.com/download/server>.

During installation:

- create your personal user (the one you will ssh in as); nothing else is needed;
- **install the OpenSSH server** when the installer offers it: it is the only way to administer the machine without a monitor and keyboard attached;
- do not install extra snaps (Docker, Kubernetes and the like): they are not needed.

## 1.2 Checking your starting point

Connect over SSH and look at what you have:

```
ssh your-user@your-server-address
```

```
lsb_release -a          # Ubuntu version
python3 --version       # must be >= 3.11
free -h                # available memory
df -h /                 # space on the system disk
ip -brief address       # network addresses
```

**Check.** `python3 --version` must answer Python 3.12.x (or higher) and `df -h /` must show at least 5 GB free. EMMA takes up a few tens of MB, but the virtualenv, the logs and the backups want room to breathe.

## 1.3 Updating the system

Before installing anything, bring the system up to date:

```
sudo apt update
sudo apt upgrade -y
sudo reboot          # only if the upgrade touched the kernel
```

If apt reports that a restart is required (\*\* System restart required \*\*), restart now: a half-applied kernel is the stupidest cause of later problems.

## 1.4 System packages

All from the official Ubuntu repositories. **We add no PPA and no external repository**, and it is worth saying why: every third-party repository is a party that can publish packages onto your machine with root privileges, and it has to be kept updated and trusted for years. It is not needed here: everything EMMA requires is in the official repositories, and the Python libraries come from PyPI inside an isolated virtualenv, without touching the system Python.

```
sudo apt install -y \
    python3 \
    python3-venv \
    python3-pip \
    git \
    curl \
    ca-certificates \
    sqlite3 \
    tar \
    gzip
```

What each one is for:

| Package         | Why                                                                   |
|-----------------|-----------------------------------------------------------------------|
| python3         | the interpreter; already present, listed for completeness             |
| python3-venv    | creates the project's isolated virtual environment                    |
| python3-pip     | installs the dependencies inside that environment                     |
| git             | fetches the code and lets you go back to an earlier version           |
| curl            | manual checks (the /health endpoint, connectivity)                    |
| ca-certificates | root certificates for the HTTPS connections to Anthropic and Telegram |
| sqlite3         | used by scripts/backup.sh for a consistent snapshot of the database   |
| tar, gzip       | used by scripts/backup.sh (normally installed already)                |

**Check.**

```
python3 -m venv --help > /dev/null && echo "venv ok"
git --version
curl -sI https://api.anthropic.com | head -1
```

The last command must return an HTTP/... line: any answer will do, it means the server reaches the internet over HTTPS. If nothing answers, the problem is networking or DNS and has to be solved before going on.

## 1.5 A dedicated system user

EMMA must not run as you, and certainly not as root. It gets a user of its own, **with no ability to log in and no privileges**: if one day a tool has a bug or a dependency turns out to be malicious, the damage stays confined to what that user can touch — which is almost nothing.

```
sudo useradd --system --create-home --home-dir /opt/emma --shell /usr/sbin/nologin emma
```

What the options do:

- `--system`: a service user, with a low UID, excluded from login lists;
- `--create-home --home-dir /opt/emma`: the home directory is the installation directory, so the service has a place of its own to live in;
- `--shell /usr/sbin/nologin`: nobody can log in as emma, not even with the right password, because no password exists.

**Check.**

```
id emma
# uid=999(emma) gid=999(emma) groups=999(emma)
sudo -u emma whoami
# emma
```

## 1.6 The installation directory and its permissions

```
sudo mkdir -p /opt/emma
sudo chown emma:emma /opt/emma
sudo chmod 750 /opt/emma
```

750 means: emma reads and writes, the emma group reads, and every other user on the machine cannot even see the contents. Given that the `.env` file with your API key ends up in there, this is the minimum.

To work comfortably, add yourself to the emma group:

```
sudo usermod -aG emma $USER
newgrp emma          # applies the new group to the current session
```

Without this step you would have to prefix every command inside `/opt/emma` with `sudo -u emma`. Note that `newgrp` applies only to the current shell: on your next SSH connection the group will be active on its own.

## 1.7 The second disk, for backups

You think in terms of a “D drive”, which on Windows is a letter; on Linux a second physical disk is a device to be mounted at a point in the directory tree. The result is the same — data on a disk other than the system one — but the procedure is this.

**If you do not have a second disk**, skip ahead to chapter 2: there is nothing to configure. `backup.sh` picks its destination by itself, with this rule:

| Situation                                            | Where it writes                       |
|------------------------------------------------------|---------------------------------------|
| /mnt/backup really is a separate disk                | /mnt/backup/emma                      |
| there is no second disk mounted                      | /var/backups/emma, on the system disk |
| you set <code>BACKUP_DIR</code> in <code>.env</code> | there, regardless, without arguing    |

The backup **always happens**: an archive in a mediocre place is worth more than an archive that does not exist. When it falls back to the system disk it says so in the log and in the manifest, because it remains a compromise — it protects you from your own mistakes, not from the disk failing.

**Why it does not write to /mnt/backup when the disk is not mounted.** It would appear to work, and would instead fill the system disk without saying so; worse, the day you did mount the disk for real those archives would disappear underneath the mount point, still occupying space that nobody can see any more. This is why the script checks that it is a separate filesystem, not that the directory exists.

### 1.7.1 Identifying the disk

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT,MODEL
```

Example output:

```
NAME    SIZE TYPE FSTYPE MOUNTPOINT MODEL
sda     240G disk                KINGSTON_SA400
├─sda1   1G part vfat    /boot/efi
└─sda2  239G part ext4    /
sdb     500G disk                WDC_WD5000AAKX
```

Here sda is the system disk (it contains /) and sdb is the second disk, still without a filesystem and without a mount point. **Check the letter twice before going on:** the commands that follow erase the contents of the disk you name.

### 1.7.2 Partitioning and formatting

**Warning: the two commands below destroy all data on /dev/sdb.** If the disk holds anything you need, copy it elsewhere first.

```
sudo parted /dev/sdb --script mklabel gpt mkpart primary ext4 0% 100%
sudo mkfs.ext4 -L backup /dev/sdb1
```

ext4 is the right choice here: stable, supported everywhere, with no options to understand. The backup label is used in a moment.

### 1.7.3 Mounting it permanently

A mount by hand disappears on reboot. To be permanent it has to be written into /etc/fstab, and it has to be written using the filesystem's **UUID**, not /dev/sdb1: device names can change between boots if you add or remove a disk, the UUID does not.

```
sudo mkdir -p /mnt/backup
sudo blkid /dev/sdb1
# /dev/sdb1: LABEL="backup" UUID="1a2b3c4d-..." TYPE="ext4"
```

Copy the UUID and add the line to /etc/fstab:

```
sudo cp /etc/fstab /etc/fstab.bak          # a safety net
echo 'UUID=1a2b3c4d-... /mnt/backup ext4 defaults,nofail 0 2' | sudo tee -a /etc/fstab
```

The **nofail option matters**: without it, if one day the disk fails or you unplug it, the server does not finish booting and sits in emergency mode — with the assistant switched off because of a backup problem. With nofail it carries on and the backup fails with a clear message in the logs, which is the right behaviour.

```
sudo systemctl daemon-reload
sudo mount -a
```

**Check.**

```
findmnt /mnt/backup
df -h /mnt/backup
```

If mount -a reports no errors and findmnt shows the disk, the fstab line is correct and the mount will survive a reboot. An error here has to be fixed now: a wrong fstab stops the machine from booting.



### 1.7.4 The backup directory

```
sudo mkdir -p /mnt/backup/emma
sudo chown emma:emma /mnt/backup/emma
sudo chmod 700 /mnt/backup/emma
```

700 because the archives contain the `.env`, and therefore your API key: only the emma user should be able to read them. This path is the one you will write into `BACKUP_DIR` in chapter 4.

## 1.8 Firewall

EMMA **exposes nothing inbound**: it talks to Telegram and to the LLM provider by opening outbound connections, and the `/health` endpoint listens only on `127.0.0.1`, reachable from the machine itself and nowhere else. The firewall therefore has nothing to open for EMMA: it only has to let you in over SSH.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw enable
```

**Before running `ufw enable`, make sure `sudo ufw allow OpenSSH` succeeded.** Enabling the firewall without having opened SSH locks you out of the machine, and getting back in needs a monitor and a keyboard.

**Check.**

```
sudo ufw status verbose
```

You must see `Default: deny (incoming), allow (outgoing)` and a single allowed rule, `22/tcp (OpenSSH)`. This is exactly the configuration the architecture implies: no open port, no webhook, no certificate to manage.

## 1.9 The credentials you need

Before deploying, obtain these three values. Keep them to one side: you will write them into the `.env` in chapter 4.

**The LLM provider key** — one of the two, depending on which you will use.

*Anthropic (paid).* At <https://console.anthropic.com>, under *API Keys*, create a key. It starts with `sk-ant-`. It is shown **once only**: copy it straight away. Remember also to set a monthly spending limit in the billing section — it is the simplest protection against a surprise.

*Groq (free tier).* At <https://console.groq.com> create a key: it starts with `gsk_`. This is the option to choose if you want to keep the cost at zero; in exchange, the available models depend on the plan and the per-minute request limits are tighter. You can change provider at any time — it is one line in the `.env`.

**The Telegram bot token.** From your phone, write to [@BotFather](#):

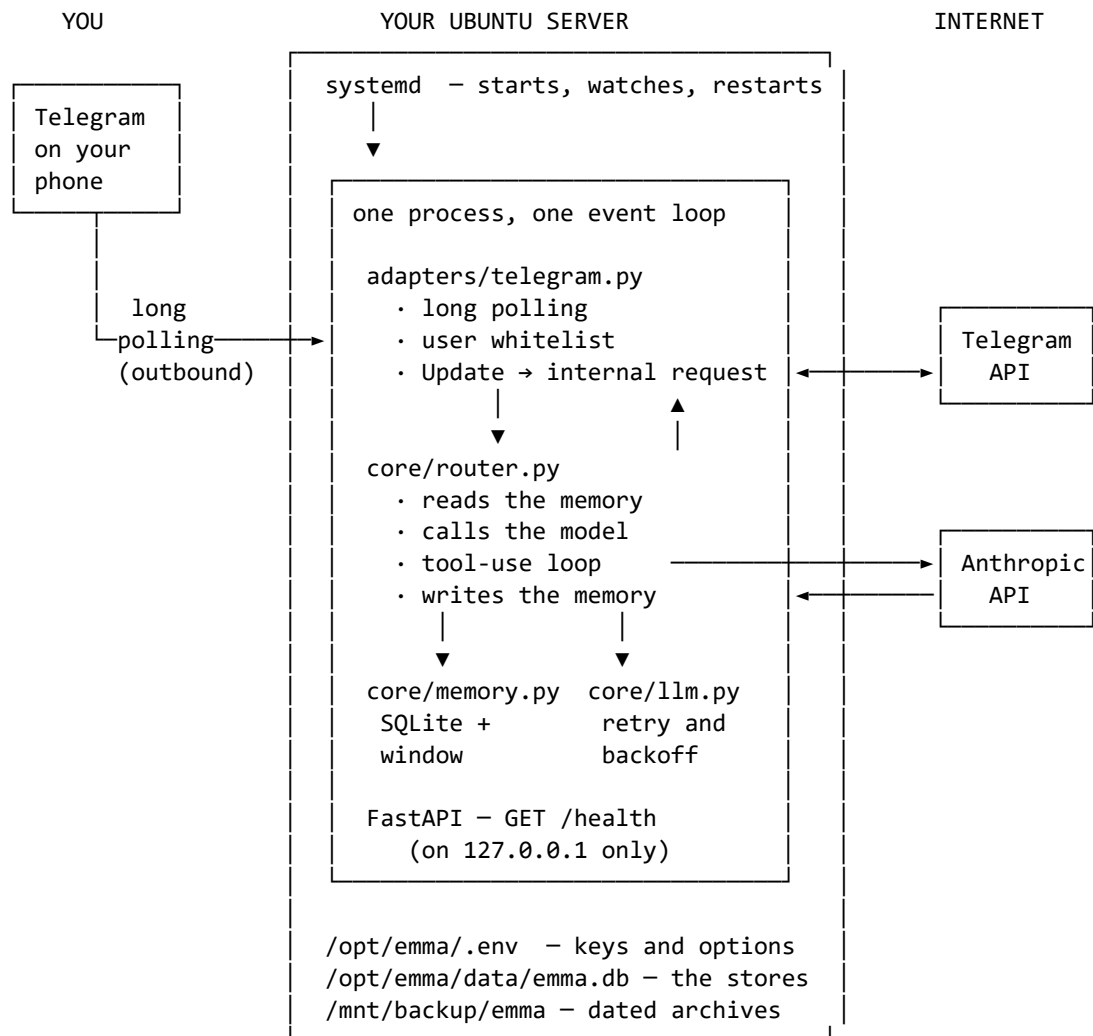
1. `/newbot`
2. choose a display name (Emma, for instance)
3. choose a username ending in `bot` (`emma_assistant_bot`, for instance)

BotFather answers with a token of the form `123456789:AAH...`. **It is a credential**: whoever holds it controls the bot. It never goes into Git, never into a message, never into a log.

**Your Telegram user ID.** It is not the `@username`, it is a number. Write to [@userinfobot](#): it answers with your Id. That number goes into `TELEGRAM_ALLOWED_USER_ID` and is what makes the bot answer you and nobody else.

# Chapter 2 — Architecture

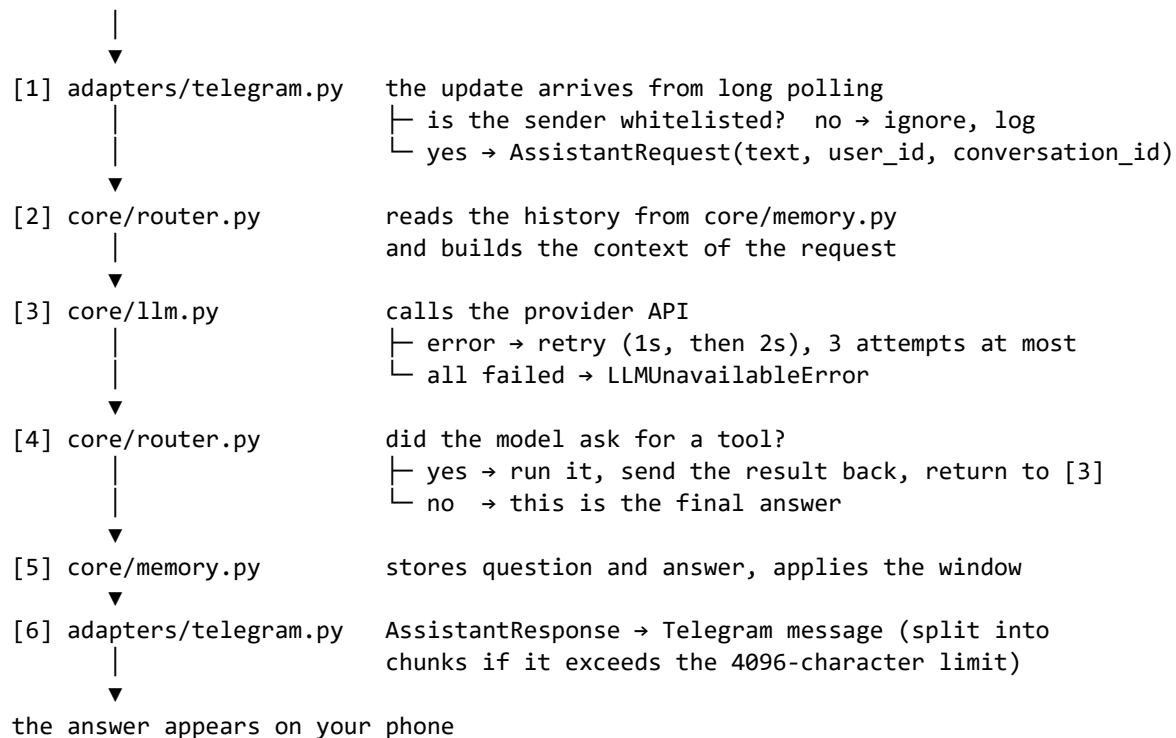
## 2.1 The overall picture



No inbound port. Every arrow to the internet starts on the inside.

## 2.2 The path of a message

you write "what is the weather like?"



If [3] fails outright the router does not raise: it answers with a sentence saying which of the faults happened -- model unreachable, quota exhausted, empty answer, too many tool rounds -- stores nothing, and the process stays alive.

## 2.3 The adapter pattern: why core/ does not know what Telegram is

This is the most important structural decision in the project. There is a single rule: **no file under core/ imports Telegram, and no Telegram concept (chat id, update, message formatting) enters the core.**

The boundary is made concrete by two tiny objects in core/router.py:

```
AssistantRequest(text: str, user_id: str, conversation_id: str)
AssistantResponse(text: str, degraded: bool = False)
```

The adapter translates in both directions: from a Telegram Update to an AssistantRequest, and from an AssistantResponse to a message sent in the chat.

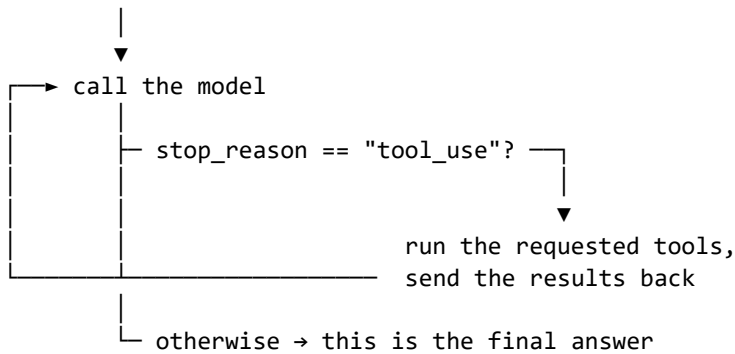
Why it matters: when the voice satellite on the Raspberry Pi arrives, transcribed speech will become an AssistantRequest with exactly the same contract. The router, the memory and the model client **will not change by one line**. If the router read update.message.chat.id instead, every new channel would mean going back into it, and every change would risk breaking the channel that already works.

The cost of this discipline is a dozen lines of conversion. The benefit is that the later phases of the roadmap are built on top of it rather than inside it.

## 2.4 The agentic router: the loop, and what it now has to call

An assistant that can only answer in words is a chatbot. One that can *do* things has to be able to call tools, look at what they returned and decide the next step. The API protocol for this is called tool use, and it has the shape of a loop:

```
build the context
```



From v0.1 to v0.2 the tool list stayed **empty**: the model could ask for nothing and the loop always exited on the first round. The code was all there, though, and tested — precisely so that this moment would arrive without having to write it.

From **v0.3** the first tools were registered, and by v0.4 there are eleven (section 3.3bis). `core/router.py` was not touched by one line to add any of them. Adding another means writing a class with four attributes and handing it to the router:

```
class Clock:
    name = "current_time"
    description = "Returns the current time."
    input_schema = {"type": "object", "properties": {}}

    async def run(self, arguments: dict) -> str:
        return datetime.now().strftime("%H:%M")
```

```
router = Router(llm=llm, memory=memory, system_prompt=prompt, tools=(Clock(),))
```

`core/router.py` is not touched. That was the point.

Two protections are already inside the loop, because without them it would be fragile: a ceiling on the number of rounds (`max_tool_iterations`, five), otherwise a model that keeps asking for tools would generate an unbounded sequence of billable calls; and containment of tool errors, which are returned to the model as a result with `is_error` instead of propagating — a faulty tool must not be able to bring the turn down.

## 2.5 Memory behind an interface

`core/memory.py` defines an abstract interface with three operations:

```
async def get_history(conversation_id) -> list[StoredMessage]
async def append(conversation_id, message) -> None
async def prune(conversation_id) -> None
```

and two implementations: `InMemoryConversationMemory` (a dictionary in RAM, lost on restart, used in the tests) and `SQLiteConversationMemory` (a SQLite file via `aiosqlite`, persistent across restarts, live in production since v0.2). Both share the same sliding window of `MAX_HISTORY_MESSAGES` messages per conversation.

The value of the interface showed itself when v0.2 landed: the router did not change by one line. The router's tests run against the real memory (SQLite on a temporary file) and check the actual contract, independently of the backend.

Two non-obvious details of the current implementation:

- **the methods are asynchronous** even though today they do no I/O. A database will, and changing the signature from synchronous to asynchronous later would mean touching the router: exactly what the interface exists to avoid.

- **the window never starts with an assistant message.** The Messages API rejects a conversation that does not begin with the user; if the cut left an answer at the head, one further message is dropped. Without this rule, an odd value of MAX\_HISTORY\_MESSAGES would break the system at random after a few exchanges.

## 2.6 Resilience: three levels

The assistant has to fail well. There are three overlapping safety nets, each for a different kind of fault.

**Level 1 — the model call** (`core/llm.py`). Three attempts with exponential backoff: immediately, after 1 second, after 2. It absorbs short disturbances — a lost packet, a 529 from an overloaded API — without you noticing. The SDK's own retries are disabled (`max_retries=0`), because otherwise the real number of attempts would be nine, with multiplied waits. Only what is worth retrying is retried: a 401 for a wrong key fails on the first try, because retrying would not make the key correct.

**Level 2 — the turn** (`core/router.py`). If every attempt fails, the exception does not travel up: it becomes a courteous answer. The process stays alive and the failed turn is not stored — otherwise the conversation would fill with apologies that the model would later try to explain.

The sentences are not interchangeable, because the faults are not:

| Reason (in the log)            | What you receive                                                 | Why it differs                                                    |
|--------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------|
| <code>model_unreachable</code> | <i>"Non riesco a contattare il cervello... riprova tra poco"</i> | retrying makes sense                                              |
| <code>quota_exhausted</code>   | <i>"Ho raggiunto il limite... riprova fra circa 11 minuti"</i>   | retrying at once does <b>not</b> , and the server states the wait |
| <code>empty_answer</code>      | the model answered with no text                                  | rare, usually a turn closed on a tool block                       |
| <code>tool_loop_ceiling</code> | too many tool rounds                                             | protects against an endless loop                                  |

(The sentences are quoted as EMMA actually says them; she speaks Italian to her owner. See rule 6 in `CLAUDE.md`.)

A **database** fault is handled here too, and does not stop the answer: if the history cannot be read the turn goes ahead without context, and if it cannot be written the answer is sent anyway (it had already been paid for in tokens). In both cases the log says so at ERROR level.

**Level 3 — the process** (`systemd/emma.service`). If the process really dies — an unforeseen bug, the OOM killer, a machine restart — `Restart=always` brings it back after 5 seconds. With a limit: five failures in five minutes and `systemd` stops, because at that point the cause is not transient (typically a wrong `.env`) and restarting forever would hide the problem instead of solving it.

## 2.7 Why FastAPI if there is nothing to expose

A fair question: the bot works by long polling and receives no HTTP requests. Why a web server?

Two reasons, one immediate and one for later. The immediate one is the `/health` endpoint on `127.0.0.1:8000`: a `curl` says whether the process is alive, which model it is using and — since 0.3.0 — **whether it is actually well**, without having to read the logs. The later one is the voice satellite on the Raspberry Pi, which will have to talk to the central node over HTTP: when it arrives, the server is already there and startup will not have to be rethought.

Up to 0.2.x the endpoint answered `"status": "ok"` under all circumstances, a dead database included: a check that cannot report anything is a liveness probe with the wrong name. Now, before answering, it really reads from the store — the same operation every turn depends on, far cheaper than a `PRAGMA integrity_check` — and if it cannot, it answers 503 with `"status": "degraded"`, so that even an automated check that cannot read JSON understands. Alongside it publishes the turn count, how many were degraded, the last reason and how long ago:

```
curl -s http://127.0.0.1:8000/health | python3 -m json.tool
```

| Field                  | What it is for                                                              |
|------------------------|-----------------------------------------------------------------------------|
| status                 | ok or degraded: it is degraded if either of the two below is not right      |
| store                  | is the database answering? with the kind of error if not                    |
| telegram               | listening or not polling — see below, it is the most treacherous blind spot |
| version / commit       | which code is actually running, not which should be                         |
| turns / degraded_turns | how often an answer was a fallback                                          |
| last_degraded_reason   | which of the four faults, by name                                           |
| seconds_since_degraded | how long ago: null means never since it started                             |

**The bot can go deaf without dying.** It is the most treacherous fault of all, and you saw it for real on 31 August: the process is alive, uvicorn answers, the database is fine — but Telegram long polling has stopped, and nobody can talk to her any more. From the phone it is indistinguishable from a bot that is switched off. Up to 0.3.0 /health knew nothing about it and answered ok. Now the telegram field says whether updates are still arriving, and if they are not the whole status becomes degraded with a 503 — even when everything else is fine, because an assistant that cannot hear you is not healthy.

**Who asks it.** Since 0.3.0 scripts/backup.sh does, that is, the job that runs every night at 03:30 anyway — and it is the right place for another reason too: it has just copied that service’s database, so it has a motive of its own for wanting to know its state. The outcome goes both into the journal and into the MANIFEST.txt inside the archive, so that a restored backup also says whether the service was well at the moment it was taken:

```
git commit: 7e5d1fc (v0.4.0, deployed 2026-09-03T01:30:56+02:00; from the VERSION stamp, not a checkout)
database:   emma.db (consistent snapshot, integrity verified)
service:    ok - {"status":"ok","store":"ok","telegram":"listening",...}
```

A service that is stopped or degraded **does not make the backup fail**: a halted process is a reason to keep the data, not to skip it. It is written down in plain words, though:

```
journalctl -u emma-backup | grep WARNING | tail
```

FastAPI and Telegram polling share **a single event loop**: uvicorn owns the loop, and the Telegram adapter is started and stopped by the application’s *lifespan*. This means a systemctl stop emma closes polling cleanly instead of killing it halfway through an update.

REVISIONE.md holds the critical discussion of this choice, together with the alternative that has no web server.

## 2.8 The provider and the model

EMMA supports two LLM backends, selected through the LLM\_PROVIDER variable:

| Value               | Backend                          | Cost                |
|---------------------|----------------------------------|---------------------|
| anthropic (default) | Claude via the Anthropic API     | paid per token      |
| groq                | Llama / GPT-OSS via the Groq API | free tier available |

### Anthropic

The default is **Claude Sonnet 4.6** (claude-sonnet-4-6). ANTHROPIC\_MODEL accepts any valid identifier — change the line in .env and restart the service, with no change to the code.

## Groq (free tier)

Set LLM\_PROVIDER=groq and supply a free key from [console.groq.com](https://console.groq.com). The default model is openai/gpt-oss-120b; you can change it with GROQ\_MODEL. Which models are available depends on the account's plan — to list the accessible ones:

```
curl -s -H "Authorization: Bearer $GROQ_API_KEY" \
  https://api.groq.com/openai/v1/models | python3 -c \
  "import json,sys; [print(m['id']) for m in json.load(sys.stdin)['data']]"
```

# Chapter 3 — Implementation

This chapter explains the code file by file: what each module is for, how it is built and why it is built that way. It is for you, to maintain it, and for anyone who wants to contribute, to find their bearings.

## 3.1 The map

|                          |                                                        |
|--------------------------|--------------------------------------------------------|
| emma/                    |                                                        |
| ├─ main.py               | startup: FastAPI + polling in the same event loop      |
| ├─ config.py             | reads and validates .env                               |
| ├─ adapters/             |                                                        |
| │   └─ telegram.py       | the only file that knows what Telegram is              |
| ├─ core/                 |                                                        |
| │   └─ router.py         | orchestrator: context → model → tools → answer         |
| │   └─ llm.py            | Anthropic/Groq client, retry and backoff               |
| │   └─ memory.py         | memory interface + RAM and SQLite implementations      |
| │   └─ tasks.py          | the queue of commissioned development jobs             |
| │   └─ version.py        | which commit is running, and whether it drifted        |
| │   └─ retry.py          | the backoff policy, stated once                        |
| ├─ tools/                |                                                        |
| │   └─ development.py    | the four tools EMMA commissions development with       |
| │   └─ facts/            | facts that do not expire                               |
| │   └─ clock.py          | the time where you are                                 |
| │   └─ introspection.py  | the version, and the inventory of her own tools        |
| │   └─ toolstate/        | switching a tool off, and two-stage removal            |
| ├─ prompts/              |                                                        |
| │   └─ system_prompt.txt | EMMA's personality                                     |
| ├─ scripts/              | backup, deploy, and the development-queue scripts      |
| ├─ systemd/              | the service and the backup timer                       |
| ├─ tests/                | the pytest suite, entirely offline                     |
| ├─ data/                 | (not in Git) the SQLite database and its two snapshots |
| └─ docs/                 | this guide, and the diagrams in the README             |

The dependency always runs in the same direction: main.py knows everyone, adapters/ knows core/, **core/ knows nobody** but itself.

## 3.2 config.py — the configuration

It exposes two things: the immutable Config dataclass, and load\_config(), the only supported way to build one.

The principle is **fail immediately and clearly**. Every missing mandatory variable, every malformed number, every unreadable personality file raises a ConfigError naming the guilty variable. If the process gets past startup, the configuration is valid and no other module has to check it again.



Details worth knowing:

- **real environment variables win over the `.env`** (override=False). This is what lets systemd, or a test, override a single setting without editing a file.
- **relative paths are anchored to the project directory**, not to the working directory: `SYSTEM_PROMPT_PATH=prompts/system_prompts` behaves identically whether you start the process from `/opt/emma` or from `/`.
- **BACKUP\_DIR and BACKUP\_KEEP are not used by the application** — `scripts/backup.sh` reads them on its own account. They are validated here all the same, so that a `BACKUP_KEEP=zero` is discovered when the service starts and not at half past three in the morning.
- **the personality file is read at startup** on purpose, to turn a wrong path into a configuration error rather than a surprise on the first message.

### 3.3 core/memory.py — the memory

ConversationMemory is the abstract class with the three methods seen in chapter 2. StoredMessage is the role/text pair that travels through it.

The module provides two concrete implementations.

**InMemoryConversationMemory** uses a dictionary of lists in RAM. It is used in the tests (fast, no I/O) but loses everything when the process restarts.

**SqliteConversationMemory** (live in production since v0.2) persists messages to a SQLite file via `aiosqlite`. It has to be opened with `open()` at startup and closed with `close()` at shutdown — the FastAPI lifespan takes care of that. The file path is controlled by `MEMORY_DB_PATH` (default `data/emma.db`); the directory is created automatically if it does not exist.

#### Self-repair

On opening, EMMA runs `PRAGMA integrity_check` against the database. If it **fails**:

1. the damaged file is **moved**, never deleted, to `emma.db.corrupt-<date>`, along with its `-wal` and `-shm` files (an old write-ahead log must not end up on top of the restored database);
2. the most recent snapshot that passes the same check is put in its place; if that one is unreadable too, the previous generation is tried;
3. if no snapshot is usable, EMMA starts again with an empty history;
4. **every step is logged at ERROR level**, with the path of the file that was set aside.

Snapshots are written with `VACUUM INTO` — which produces a consistent copy of a database that is in use, something an ordinary file copy does not guarantee — on every successful start and every clean shutdown. Two generations are kept (`emma.db.snapshot` and `.snapshot.prev`) and each is verified before it replaces the previous one.

The database uses `journal_mode=WAL`, far more resistant to a brutal interruption (a kill, the OOM killer, a power cut) than the default journal.

**Recovery only triggers on established corruption.** If EMMA fails to start for another reason — an incomplete `.env`, a missing dependency, a code error — this code is never even reached, and that is deliberate: restoring a database because something else broke would throw away good history without fixing the real fault. The full reasoning is in entry 16 of `REVISIONE.md`.

Three implementation choices common to both:

- **an `asyncio.Lock` protects the read-modify-write sequences.** PTB can run two handlers concurrently, and without the lock two messages close together could corrupt the window.
- **`get_history` always returns a copy**, so that whoever receives it can manipulate it without accidentally changing shared state.
- **`prune` is idempotent**: calling it twice in a row changes nothing. This exists so it can be called freely, without having to wonder whether it has been done already.

### 3.3bis core/tasks.py and tools/ — commissioning development

EMMA cannot modify her own code: she is the running process. She can however **record that a change is needed**, and report back on how it is going. This is the mechanism introduced in v0.3; the full reasoning, including what was deliberately left out, is entry 17 of REVISIONE.md.

core/tasks.py is the queue. A job passes through six stages — new, understood, implemented, committed, pushed, deployed — and at each transition it stops and waits for an answer from you. The stage records what is *done*; the note asks your permission for the next step.

There are eleven tools in all. Four of them are in tools/development.py:

| Tool                | When it is used                                  |
|---------------------|--------------------------------------------------|
| request_development | you record a request for a change                |
| work_status         | you ask how the jobs are going                   |
| answer_question     | you answer a pending question                    |
| abandon_development | you want a job you no longer need out of the way |

Two are in tools/facts/, the persistent-memory module:

| Tool          | When it is used                                        |
|---------------|--------------------------------------------------------|
| remember_fact | you ask her to remember something that must not expire |
| forget_fact   | you ask her to forget it                               |

One is in tools/clock.py:

| Tool         | When it is used                     |
|--------------|-------------------------------------|
| current_time | you ask what time or what day it is |

Two talk about herself, in tools/introspection.py:

| Tool            | When it is used                                           |
|-----------------|-----------------------------------------------------------|
| running_version | you ask which version is running                          |
| list_tools      | you ask what she can do, how many tools she has, or which |

And two are for taking a tool out of the way, in tools/toolstate/:

| Tool        | When it is used                                         |
|-------------|---------------------------------------------------------|
| remove_tool | you want a tool gone — <b>in two stages</b> , see below |
| enable_tool | you want to switch a disabled one back on               |

**Why two stages.** Taking a tool out of the code is a development job, and it is not undone quickly. So the first time you ask, the tool is **only disabled**: it disappears from her capabilities immediately — not at the next restart, from the next message — and you can switch it back on whenever you like. If you ask a second time **while it is still off**, then she files the job to actually remove it from the codebase.

The second step is not a formality, and it is **measured**: at least an hour must have passed since it was switched off. Without that threshold, “two requests” would be nothing but a counter — a turn allows several tool rounds, so she could switch a tool off and ask for its removal in the same breath, having learnt nothing in between. With the threshold, “already off” really does mean you have done without it.

If you ask too soon she tells you, and tells you how much longer. The hour is a choice: long enough that no single conversation crosses it, short enough that someone who has genuinely decided does not have to wait until tomorrow. It is changed in one line (`MIN_TIME_OFF_SECONDS`).

And asking a third time does not open a second job: she answers with the number of the one that already exists.

Two tools cannot be switched off, `list_tools` and `enable_tool`: without the first you would not know what is off, without the second you could not switch it back on — and the only way out would be editing by hand on the server.

A disabled tool is still listed by `list_tools`, marked (*disattivato*). If it vanished from the list you would no longer know what to ask to switch back on.

**Why a tool is needed to list the tools.** It seems absurd that she has to ask: we hand her the tools with every request. But the declarations reach the model through the API’s dedicated field, as *functions it may call*, not as data it can read — so enumerating them is not something it can do reliably about itself. Asked how many tools she had, she could not answer.

`list_tools` receives **the same tuple the router receives**, itself included: a hand-written list would be a second place to update, and the first to be forgotten.

**There is no third tool for reading the facts back**, and that is deliberate: they are all in front of the model on every turn already, so a tool to go and fetch them would answer a question whose answer it can already see — and every declaration is paid for on every message, including the ones where you only wrote “hello”. This module’s two declarations cost about 303 tokens per turn.

The module is installed and uninstalled with **two lines in `main.py`**: the one that builds `FactStore` and the one that hands the tools and the context provider to the router. `core/` does not know what a fact is, exactly as it does not know what a development job is.

**Abandoning does not delete.** The row stays in the database, marked abandoned and with the reason you gave: a job removed by mistake is still readable, and a decision taken in one message can be understood a week later. It is the same choice made for a corrupt database, which is quarantined and never removed. Only **open** jobs can be abandoned: removing one that is already finished would rewrite the history of what was asked, not undo any work. The prompt asks EMMA to tell you which job she is about to abandon and to wait for confirmation, unless you were the one who gave the number.

Two properties worth knowing, because they explain why it is built this way:

- **EMMA never speaks first.** No line of this code sends notifications. The questions stay in the queue and you see them when you ask; your answer travels back the same way.
- **The queue lives in the same SQLite file as the memory.** This is not laziness: the integrity check, the snapshots and the consistent backup built around that file therefore cover the jobs too. A second database would have been left uncovered, and silently so.

## The state EMMA always has in front of her

`DevelopmentContext` is not a tool: it is a **context provider**. The router asks it on every turn and appends a line to the system prompt:

Stato dei lavori di sviluppo in questo momento: 2 aperti (#1, #2). Di questi, 2 attendono una risposta dell'utente (#1, #2). Questa riga e' sempre aggiornata: se la conversazione precedente dice un numero diverso, quella e' vecchia e questa ha ragione.

(The line is in Italian because it is written for the model that answers in Italian; it says how many jobs are open, how many await your answer, and that this line is always more recent than anything in the conversation.)

It exists for a reason discovered in the field: **a tool is consulted only if the model decides to consult it**, and that decision can go wrong. It happened — EMMA repeated, word for word, a wrong answer from a quarter of an hour earlier without going back to check. Persistent memory and tools damage each other: an answer derived from a tool, once stored, becomes indistinguishable from a fact.

Measured over ten attempts: 6 correct out of 10 with poisoned memory and no context, 10 out of 10 with clean memory and the context active. The full reasoning, and the alternatives rejected, are in entry 17.10 of `REVISIONE.md`.

The point is not the number — which holds for *this* model — but that a line which is always present requires no decision, so the behaviour does not quietly get worse the day you change provider.

At the other end of the queue there are two shell scripts, described in section 4.9: `scripts/task-queue.sh` on the server, the only thing the development key is authorised to run, and `scripts/watch-tasks.sh` on the PC, which waits without consuming anything.

The `dev_heartbeat` table records when a development session last looked at the queue. It exists because there is no service behind it that restarts by itself: if the session dies, jobs pile up without anyone complaining, and the absence of a heartbeat is the only way to notice. `work_status` tells you.

### 3.4 `core/llm.py` — the model client

It is the only file that imports the provider SDKs (`anthropic` and `groq`). It contains two classes with the same interface — `AnthropicLanguageModel` and `GroqLanguageModel` — chosen in `main.py` according to `LLM_PROVIDER`. The router does not know which of the two it is using. Both do three things.

**They hide the SDK.** Responses are converted into our own types — `TextBlock` and `ToolUseBlock` inside an `LLMResponse` — so the router does not depend on the SDK's objects. Blocks of an unknown type are ignored rather than raising: a future addition to the API must not be able to stop a running assistant.

**They apply the retry policy.** Three attempts, waits of 1s and 2s, a 60-second timeout per request. Every failed attempt produces a log line with the kind of error; a success produces one with `stop_reason` and the tokens consumed in and out — that is where you see what the assistant really costs.

**They translate final failure** into `LLMUnavailableError`, which is what the router catches in order to answer courteously. Only transient errors (connection problems, 5xx) are retried: a permanent 4xx — wrong key, malformed request — fails at once, without burning three seconds on useless attempts.

One method deserves attention: `to_assistant_message()`. The Messages API is *stateless*, so to continue an agentic turn the model's previous answer has to be sent back word for word, tool blocks included. That method rebuilds it in the right shape.

### 3.5 `core/router.py` — the orchestrator

The heart of it. It holds the boundary objects (`AssistantRequest`, `AssistantResponse`), the `Tool` protocol and the `Router` class.

`handle()` runs a complete turn and **never raises** an exception caused by the model or by a tool: any fault becomes a degraded but polite answer. The fallback messages (model unreachable, quota exhausted, empty answer, too many rounds) are constants at the top of the file. Like the strings the tools return, they are written in Italian, because they are the ones you read and a programmer does not.

`_run_agentic_loop()` is the tool-use loop. `_execute_tool()` runs a single tool and captures its exceptions, returning them to the model as an error result.

A behavioural rule worth knowing: **degraded turns are not stored**. If the model was unreachable, your question and the courteous answer both disappear, so the next message starts from a clean history instead of from an apology.

## 3.6 adapters/telegram.py — the channel

It builds the `python-telegram-bot` Application, registers a single handler for text messages and an error handler that logs any exception that escapes, keeping the bot standing.

Points worth knowing:

- **the whitelist is an explicit check in the handler**, not a PTB filter, because this way an attempt by a stranger leaves a WARNING line in the logs. If someone finds your bot, you find out.
- **drop\_pending\_updates=True at startup**. After a restart you get a live assistant, not a burst of answers to questions from three hours ago.
- **long answers are split**. Telegram rejects messages over 4096 characters; the cut prefers a newline near the limit, so as not to break lines and paragraphs in half.
- **the “typing” indicator** is sent before calling the model, so that the two seconds of waiting do not look like silence.
- **commands (/start and the like) are ignored**. This is a deliberate simplification: the only interaction is writing in natural language.

## 3.7 main.py — startup

It is the *composition root*: the only place where the concrete classes are chosen.

```
llm      = AnthropicLanguageModel(...)      # or GroqLanguageModel, per LLM_PROVIDER
memory   = SqliteConversationMemory(...)    # persistent since v0.2
router   = Router(llm, memory, prompt, tools=all_tools,
                  context_providers=(DevelopmentContext(tasks), FactsContext(facts)),
                  tool_gate=tool_state)
telegram = TelegramAdapter(token, user_id, router)
```

Replacing a component is one line in this file. It is the right place to look to understand how the system is assembled.

The FastAPI *lifespan* opens the database and starts polling when the server starts, and on shutdown walks the same path backwards: it stops polling, closes the HTTP connection pool to the provider and closes the database. Logging is configured once, to stdout, in the format `timestamp | LEVEL | logger | message`, with the HTTP libraries' loggers silenced: under long polling they would emit a line every few seconds without saying anything.

`main()` returns 2 if the configuration is invalid, and logs the error without a traceback: a wrong `.env` is a usage error, and thirty lines of stack would hide the one that matters.

## 3.8 tests/ — the suite

Four hundred and ninety tests, all offline. The model is replaced by `ScriptedModel`, a fake client that returns prepared answers and records what it was asked: it implements the same interface as the real client, so it checks the actual contract.

They are spread across 24 files. The largest groups:

| File                                   | Tests | What it covers                                                                    |
|----------------------------------------|-------|-----------------------------------------------------------------------------------|
| <code>test_tools_development.py</code> | 57    | the development queue seen through the tools: filing, status, answers, abandoning |
| <code>test_rate_limits.py</code>       | 38    | 429s, retry-after, exhausted quota, and the wait the answer quotes                |
| <code>test_toolstate.py</code>         | 31    | switching off, the hour threshold, protected tools, no duplicate jobs             |

| File                   | Tests | What it covers                                                                                                                                                                                                                                        |
|------------------------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| test_router.py         | 27    | a simple turn, history, isolation between conversations, the full tool loop with the tool_result format, an unknown tool, a tool that explodes, the round ceiling, an unreachable model, an empty answer, a failed turn that does not poison the next |
| test_main_lifespan.py  | 27    | startup and shutdown, and the assertion on the exact set of registered tool names                                                                                                                                                                     |
| test_facts_store.py    | 27    | facts: limits, length, ordering, reopening                                                                                                                                                                                                            |
| test_memory_sqlite.py  | 26    | as for the memory, plus persistence across closing and reopening the database                                                                                                                                                                         |
| test_llm_groq_tools.py | 24    | the two providers described once, so they cannot drift apart                                                                                                                                                                                          |
| test_router_gate.py    | 16    | the gate: a tool switched off mid-turn stops being offered, and is refused if called                                                                                                                                                                  |
| test_clock.py          | 19    | the time zone, the Italian day and month names, the missing-tzdata fallback                                                                                                                                                                           |

The rest cover configuration, retry policy, version and drift, the Telegram adapter, the facts tools and introspection.

They do not aim at total coverage: they aim at the places where a regression would be silent.

pytest *# in the project directory, with the virtualenv active*

# Chapter 4 — Deployment

From here on the work is on the server, with the environment prepared in chapter 1.

## 4.1 The repository on GitHub

The project starts life as a Git repository. On the Windows development PC, from the project folder:

```
git init
git add .
git commit -m "initial commit: EMMA v0.1.0, text-only assistant"
git tag -a v0.1.0 -m "v0.1.0 - first working release"
```

Then create a **private** repository on GitHub (you will make it public yourself when you release) and connect it:

```
git remote add origin https://github.com/<your-account>/emma.git
git branch -M main
git push -u origin main --tags
```

**Check.** On GitHub you must see the files and you must **not** see `.env`. If you do, stop: the file has been committed, the key is compromised and has to be revoked at once on `console.anthropic.com` before anything else.

## 4.2 Fetching the code onto the server

```
sudo -u emma git clone https://github.com/<your-account>/emma.git /opt/emma
cd /opt/emma
```

If the `/opt/emma` directory already exists and is empty (we created it in chapter 1), `git clone` uses it without complaint. If it objects because it is not empty, clone into `/tmp` and move the contents.

With a private repository, `git clone` asks for credentials: use a GitHub *personal access token* in place of the password, or a deploy SSH key. Save the token with `git config --global credential.helper store` **only** if you accept that it ends up in clear text in the emma user's `~/.git-credentials`.

## 4.3 The virtual environment

```
sudo -u emma python3 -m venv /opt/emma/.venv
sudo -u emma /opt/emma/.venv/bin/pip install --upgrade pip
sudo -u emma /opt/emma/.venv/bin/pip install -r /opt/emma/requirements.txt
```

The virtualenv isolates EMMA's dependencies from the system Python: no risk of breaking Ubuntu tools that use Python, and no need for `--break-system-packages`.

**Check.**

```
sudo -u emma /opt/emma/.venv/bin/pip list | grep -Ei "anthropic|groq|aiosqlite|telegram|fastapi|uvicorn|dot"
```

You must see every library at the exact versions written in `requirements.txt`.

## 4.4 The `.env` file

```
sudo -u emma cp /opt/emma/.env.example /opt/emma/.env
sudo -u emma chmod 600 /opt/emma/.env
sudo -u emma nano /opt/emma/.env
```

600 means: only the emma user can read and write it. No other user on the machine, not even yours, can see the API key without sudo.

Fill in the mandatory values with the credentials from section 1.9.

### With Anthropic (the default):

```
LLM_PROVIDER=anthropic
ANTHROPIC_API_KEY=sk-ant-...           # your key
TELEGRAM_BOT_TOKEN=123456789:AAH...   # the token from BotFather
TELEGRAM_ALLOWED_USER_ID=123456789    # your numeric ID
```

### With Groq (free tier):

```
LLM_PROVIDER=groq
GROQ_API_KEY=gsk-...                   # a key from console.groq.com
TELEGRAM_BOT_TOKEN=123456789:AAH...
TELEGRAM_ALLOWED_USER_ID=123456789
```

And check the optional ones, which already have sensible defaults:

| Variable             | Default                   | Meaning                                 |
|----------------------|---------------------------|-----------------------------------------|
| LLM_PROVIDER         | anthropic                 | LLM backend: anthropic or groq          |
| ANTHROPIC_MODEL      | claude-sonnet-4-6         | the Anthropic model                     |
| GROQ_MODEL           | openai/gpt-oss-120b       | the Groq model                          |
| MAX_HISTORY_MESSAGES | 20                        | messages kept in the context window     |
| MEMORY_DB_PATH       | data/emma.db              | the SQLite file; created on first start |
| SYSTEM_PROMPT_PATH   | prompts/system_prompt.txt | the personality file                    |
| BACKUP_DIR           | /mnt/backup/emma          | where the archives go                   |
| BACKUP_KEEP          | 14                        | how many archives to keep               |

If in section 1.7 you mounted the disk at a different path, correct `BACKUP_DIR` now.

### Check.

```
ls -l /opt/emma/.env
# -rw----- 1 emma emma ... /opt/emma/.env
git -C /opt/emma status --short
# .env must not appear
```

## 4.5 The first start, by hand

Before installing the service, prove that everything works in the foreground, where error messages can be read immediately:

```
cd /opt/emma
sudo -u emma /opt/emma/.venv/bin/python main.py
```



You must see something like:

```
2026-08-31T14:02:10+0200 | INFO      | emma | starting emma (provider=anthropic, model=claude-sonnet-4-6, history=20 messages, db=data/emma.db)
2026-08-31T14:02:11+0200 | INFO      | adapters.telegram | telegram adapter started (long polling)
INFO:      Unicorn running on http://127.0.0.1:8000
```

On the first start the database file is created automatically inside data/.

**The data/ directory must be created before installing the service**, in section 4.6: emma.service declares it in ReadWritePaths=, and systemd **refuses to start** a unit whose ReadWritePaths points at a directory that does not exist. If you are trying it by hand as above, EMMA creates it herself.

**Now the real test: pick up your phone and write to your bot.** Within a couple of seconds you must get an answer, and the terminal must show the lines incoming message from chat\_id=... and answered chat\_id=...

If nothing happens, jump to section 6.7: the three most common reasons are a wrong token, a TELEGRAM\_ALLOWED\_USER\_ID that is not yours, and having written to a bot other than the one the token belongs to.

Stop the process with Ctrl+C.

## 4.6 The systemd service

First the database directory, which must exist **before** the unit starts:

```
sudo -u emma mkdir -p /opt/emma/data
sudo chmod 700 /opt/emma/data
```

700 because it holds your conversations. Then the service:

```
sudo cp /opt/emma/systemd/emma.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now emma.service
```

enable --now does two things at once: it starts the service now and configures it to start by itself every time the machine boots.

If you installed EMMA somewhere other than /opt/emma, edit the lines marked # PATH: and the User=/Group= pair before copying the file.

**Why the directory first.** The unit is hardened with ProtectSystem=strict, which makes the whole filesystem read-only for the service; the only exception is ReadWritePaths=/opt/emma/data, which is what lets EMMA write the history without being able to rewrite her own code. Systemd however **refuses to start** a unit whose ReadWritePaths does not exist, with an unhelpful error (Failed to set up mount namespacing). If you move the database with MEMORY\_DB\_PATH, update that line in the unit and its twin in emma-backup.service.

### Check.

```
systemctl status emma.service
```

You must read Active: active (running). Then:

```
curl -s http://127.0.0.1:8000/health
# {"status": "ok", "store": "ok", "model": "claude-sonnet-4-6", "provider": "anthropic",
#  "version": "0.4.0", "commit": "a1b2c3d", "uptime_seconds": 12.4,
#  "turns": 0, "degraded_turns": 0, "last_degraded_reason": null,
#  "seconds_since_degraded": null}
```

```
journalctl -u emma -n 30 --no-pager
```

And again the test that counts: write to the bot from your phone and check that you get an answer.

**Checking the automatic restart** (it is an acceptance criterion, and it is worth actually trying):

```
sudo systemctl kill -s SIGKILL emma.service    # simulates a brutal crash
sleep 8
systemctl status emma.service                  # must be running again
```

The logs will show it coming back. If after ten seconds the service has not returned, something in the unit is wrong: check `journalctl -u emma -n 50`.

## 4.7 The backup timer

```
sudo cp /opt/emma/systemd/emma-backup.service /etc/systemd/system/
sudo cp /opt/emma/systemd/emma-backup.timer /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now emma-backup.timer
```

The unit already covers both possible destinations (`/mnt/backup` if it is there, `/var/backups` otherwise) and creates `/var/backups/emma` itself with the right permissions, so the fallback works even on a freshly installed machine. If instead you set a `BACKUP_DIR` that is in neither, add that directory to `ReadWritePaths=` in `emma-backup.service` (the line marked `# PATH:`) before copying it: with `ProtectSystem=strict` the filesystem is read-only for the service, and without that line the backup fails with a permission denied.

If you run `scripts/backup.sh` **by hand** as the `emma` user before the timer has ever fired, the fallback directory may not exist yet: `sudo install -d -o emma -g emma -m 700 /var/backups/emma` creates it. Going through the service (`systemctl start emma-backup.service`) needs none of this.

Try it by hand straight away, without waiting for half past three:

```
sudo systemctl start emma-backup.service
journalctl -u emma-backup.service -n 30 --no-pager
```

In the log, the `destination:` line says where the archive ended up and why:

```
destination: /mnt/backup/emma [separate disk]
destination: /var/backups/emma [system disk, fallback - no separate disk available]
```

Look there, then list that directory:

```
ls -lh /mnt/backup/emma/    # or /var/backups/emma/
```

**Check.** There must be a file `emma-YYYYMMDD-HHMMSS.tar.gz` with permissions `-rw-----`. Check the contents too:

```
tar -tzf /mnt/backup/emma/emma-*.tar.gz | head -20
tar -xzOf /mnt/backup/emma/emma-*.tar.gz MANIFEST.txt
```

You must see the project files, the `.env` and the manifest with the date and the Git commit it came from.

```
systemctl list-timers emma-backup.timer
```

tells you when the next run will fire.

## 4.8 Summary: what is now on the machine

| Path                        | What it holds            | Permissions                |
|-----------------------------|--------------------------|----------------------------|
| <code>/opt/emma</code>      | code, virtualenv, prompt | <code>750 emma:emma</code> |
| <code>/opt/emma/.env</code> | API key and token        | <code>600 emma:emma</code> |

| Path                                            | What it holds                          | Permissions   |
|-------------------------------------------------|----------------------------------------|---------------|
| /opt/emma/data/emma.db                          | conversations, facts, jobs, tool state | emma:emma     |
| /opt/emma/data/emma.db.snapshot                 | verified copies for recovery           | emma:emma     |
| /mnt/backup/emma                                | dated archives                         | 700 emma:emma |
| /etc/systemd/system/emma.service                | the service                            | root          |
| /etc/systemd/system/emma-backup.{service,timer} | the backup                             | root          |

Nothing listening towards the outside; a single firewall rule, for SSH.

## 4.9 The development queue

You only need this if you want to commission development from EMMA (section 5.6). If you skip this section, EMMA still records the requests: it is simply that nobody collects them.

The mechanism is a development session, on the PC where the repository lives, reading the queue on the server. To do that it needs an SSH key — and here there is a choice worth making well.

### 4.9.1 Why a dedicated key

That session queries the server **continuously and with nobody watching**. Giving it the administration key would mean putting the most powerful credential on the machine into the one path that is unsupervised.

The dedicated key, instead, is bound to a single script, and the restriction is enforced by `sshd`, not by good intentions: whoever presents that key runs `scripts/task-queue.sh` and nothing else, whatever command they ask for. If it were stolen, the worst that can be done is writing nonsense into the job queue — which you then read — not touching the system.

The administration key stays the one you have, and is used only for deployment, which is behind a confirmation from you anyway.

### 4.9.2 On the development PC

```
ssh-keygen -t ed25519 -f ~/.ssh/emma_queue -C "emma task queue" -N ""
cat ~/.ssh/emma_queue.pub
```

Then add the destination to `~/.ssh/config`, which is **not** in the repository:

```
Host emma-queue
    HostName <your-server>
    User emma
    IdentityFile ~/.ssh/emma_queue
    IdentitiesOnly yes
```

`IdentitiesOnly yes` stops SSH from trying the other keys you have lying around first, the administration one included.

### 4.9.3 On the server

Add the public key you have just created to `/opt/emma/.ssh/authorized_keys`, preceded by the restrictions, **all on a single line**:

```
sudo -u emma tee -a /opt/emma/.ssh/authorized_keys <<'EOF'
command="/opt/emma/scripts/task-queue.sh",no-pty,no-port-forwarding,no-agent-forwarding,no-X11-forwarding
EOF
```

```
sudo -u emma chmod 600 /opt/emma/.ssh/authorized_keys
sudo -u emma chmod 750 /opt/emma/scripts/task-queue.sh
```

**Check.** From the development PC:

```
ssh emma-queue touch           # must answer: ok
ssh emma-queue list           # the queue, in JSON
ssh emma-queue whoami         # must be REFUSED
```

The last command is the one that counts: if it answers with a username instead of refused, the command= prefix was not applied and the restriction does not exist. Check again that it is all on one line and before the key.

#### 4.9.4 The permitted operations

scripts/task-queue.sh accepts only these verbs, and refuses everything else. It never accepts SQL: it builds the queries itself, and every value that goes into one is either a verified integer or a string with its quotes doubled.

| Command                      | What it does                                  |
|------------------------------|-----------------------------------------------|
| list                         | the jobs waiting for the developer, in JSON   |
| list-all                     | every job, closed and abandoned ones included |
| show <n>                     | one job                                       |
| touch                        | records that the session is alive             |
| create "<description>"       | opens a job found while working on the code   |
| advance <n> <stage> "<note>" | advances it and asks the checkpoint question  |
| finish <n> "<note>"          | closes a deployed job                         |
| abandon <n> "<note>"         | drops it                                      |

Every command also updates the heartbeat: a session that is working is alive, and it would be absurd for it to look dead because it had not called touch.

create exists for one case only: a defect discovered **while** working on the code, which would otherwise stay in the memory of whoever saw it. It does not move control — a job opened this way still stops at the first checkpoint and asks you “shall I go on?” before anything is built. The jobs that originate with you still arrive through EMMA (section 5.6).

#### 4.9.6 The three moments at which a job can be noticed

There is no service behind the queue: if nobody looks, jobs pile up. Up to 0.3.0 there was a single hook, SessionStart, which looked **when the session opened and never again** — so a job commissioned while the session was already open was noticed by nobody. It happened for real on 1 September 2026.

There are three moments, and they need different mechanisms:

| When the job arrives                        | What notices it                           |
|---------------------------------------------|-------------------------------------------|
| Before the session opens                    | SessionStart hook → queue-brief.sh        |
| With the session open, and then you write   | UserPromptSubmit hook → queue-brief.sh    |
| With the session open, and you do not write | Stop hook → watch-tasks.sh in asyncRewake |

In the .claude/settings.local.json of wherever you work from:

```
{
  "hooks": {
    "SessionStart": [
      { "hooks": [
```

```

    { "type": "command",
      "command": "bash '<path>/emma/scripts/queue-brief.sh' SessionStart 10",
      "timeout": 20 },
    { "type": "command",
      "command": "EMMA_WAKE_ON_WORK=1 POLL_SECONDS=120 bash '<path>/emma/scripts/watch-tasks.sh'",
      "asyncRewake": true, "timeout": 21600 }
  ] }
],
"UserPromptSubmit": [
  { "hooks": [ { "type": "command",
    "command": "bash '<path>/emma/scripts/queue-brief.sh' UserPromptSubmit 4",
    "timeout": 15 } ] }
],
"Stop": [
  { "hooks": [ { "type": "command",
    "command": "EMMA_WAKE_ON_WORK=1 POLL_SECONDS=120 bash '<path>/emma/scripts/watch-tasks.sh'",
    "asyncRewake": true, "timeout": 21600 } ] }
]
}
}

```

The event name is an argument because Claude Code **discards** output whose hookEventName does not match the hook that produced it. The connection timeout is the second argument: at session start, ten seconds spent finding out are free; on every message they are ten seconds you spend waiting.

It reports **only the number**, not the text of the requests: reading them would cost context in every session, including the ones where they will not be touched. If the queue is empty, or the server is unreachable and there is no cache, it prints nothing and exits successfully — a session must not fail to start, and a message must not be held up, because a machine is switched off.

**The local cache, and why it is only a safety net.** Every successful query writes the state to `~/.claude/emma-queue-state`. If the server later does not answer, that number is reported while saying explicitly that it is old and by how much: *“(the server is not answering; figure from 4 minutes ago)”*.

The order is deliberately the opposite of the obvious one: **the server first, the cache only if that fails**. Reading the cache first would make the hook instantaneous, but a cache even a few minutes old may not contain the job that was just filed — which is exactly the defect all of this exists to close. Freshness is the function; the cache buys robustness without spending any.

The hooks themselves rewrite it: every session opening and every message you send updates it, because each of those moments queries the server already. Nothing else is needed.

**A scheduled task: tried, and switched off.** On 1 September 2026 one was registered that refreshed the cache every 5 minutes even with no session open (`scripts/queue-brief.sh --refresh`). It worked, and it still had to go, for two reasons that only appeared in use.

**It was visible.** Running Git Bash as an Interactive task, it flashed a terminal window on the screen every five minutes. The `-Hidden` option of `New-ScheduledTaskSettingsSet` is not for this: it hides the task in the Task Scheduler list, not the window. Not seeing it would mean running it in session 0 (`-LogonType S4U`), which raises the question of whether the SSH keys still work from there.

**And it bought very little.** The cache is already updated on every message and every opening; the task only added refreshes while no session is open — that is, when there is nobody to tell. The only real gain was that a session opened *after* a server fault would find a slightly less stale number.

A permanent annoyance for a marginal gain is a bad trade. The task stays registered but **disabled**; it is removed entirely with `Unregister-ScheduledTask -TaskName 'EMMA queue cache'`.

#### 4.9.5 The wait that costs nothing

`scripts/watch-tasks.sh`, on the development PC, queries the queue and **exits as soon as there is something**. That is the whole trick: what waits is a shell script, which costs nothing, and the session that does cost wakes up only when there really is work. A day with no requests is a day of a sleeping shell.

```
scripts/watch-tasks.sh           # every 5 minutes, for 6 hours
POLL_SECONDS=60 scripts/watch-tasks.sh  # more responsive
```

**Since 0.3.0 it is no longer started by hand.** The `SessionStart` and `Stop` hooks launch it with `asyncRewake`, which is the mechanism Claude Code uses to wake the model when a background command **exits with code 2**. The `Stop` hook re-arms it after every turn, so it stays active for the whole session.

Two details without which it would break:

- **In hook mode the exit codes are inverted.** Normally 2 means *“I gave up, there is nothing”*; with `asyncRewake` it would mean *“wake up”*, that is, exactly the opposite. With `EMMA_WAKE_ON_WORK=1`, 2 means new work and **everything else exits 0 silently** — an alarm asserts that something is waiting, and must not be spent on a network fault.
- **It remembers what it has already announced.** Without memory, the `Stop` hook would restart it, it would find the same job still queued, and it would wake up again — forever, if that job is waiting for an answer from you. A lock also makes re-arming idempotent: asking for a watcher while one is already watching does nothing.

**It is still tied to the session.** It dies with it, and if a job arrives in the few seconds between the wake-up and the re-arm it is found on the next round. Making it genuinely guaranteed would mean a service that outlives the session: the infrastructure this project has chosen not to have.

The reliable part is the hook in section 4.9.6, which fires on every session opening and needs no running process. Together they give you: **open a session and you know at once whether there is work; while you work, if the watcher is alive it wakes you**. What cannot be promised is “I commission it at night and find it done”.

# Chapter 5 — Daily use

## 5.1 Day to day

Open Telegram, write to the bot, get the answer. That is all.

EMMA remembers the conversation: you can ask follow-up questions without repeating the context (“and tomorrow?” after asking about the weather works). The memory covers the last `MAX_HISTORY_MESSAGES` messages — with the default of 20, about ten exchanges — and **since v0.2 it survives restarts**: after a `systemctl restart` or a machine reboot the conversation picks up where you left it.

**But that window forgets by age**, and age is the wrong criterion for some things: “*my daughter is called Sara*” expires at exactly the same rate as “*what time is it*”. After a dozen exchanges it has not “faded”, it has been deleted from the database.

This is why **facts** exist. If you tell her “*remember that the home wifi is X*”, EMMA records it separately and it never expires: she will know it in a month’s time, after any restart. It works **only if you ask** — “*remember that*”, “*make a note that*”, “*don’t forget that*” — because deciding on her own what deserves to be remembered is the same class of risk as automatic summarising: it goes wrong plausibly, and nobody thinks to check.

To make her forget something, just ask. The fact is not destroyed, only set aside: as with an abandoned job and a corrupt database, a decision you can read back is less final than one you cannot.

**It costs something, and it is right to know.** The facts are put in front of the model on every single message, so they are paid for every time. Measured on real traffic: with no facts an exchange goes from ~2,360 to ~2,660 tokens, with thirty facts to ~3,100. Against Groq’s free ceiling of 200,000 tokens a day that means going from ~84 exchanges to ~64. The maximum is **50 facts**.

Typical response times: one or two seconds. The “typing” indicator appears straight away, so you know the message arrived.

## 5.2 What to expect

EMMA is a conversation with a language model, with a personality defined in `prompts/system_prompt.txt`: short answers, Italian, a direct tone, no preamble. She is good for thinking a problem through, having something explained, drafting a text, sorting out your ideas. On top of that she has eleven tools: she knows what time it is, she can keep facts that do not expire, she can tell you which version of herself is running and which tools she has, and she can be asked to commission her own development.

She does **not** have real-time internet access, or access to your files, your calendar, or your house. If you ask her for today’s weather she will say she cannot know it — and that is the intended behaviour: an honest “I don’t know” beats an invented fact. If you want one of those capabilities, that is exactly what section 5.6 is for.

## 5.3 Personalising her character

The file `prompts/system_prompt.txt` is the personality, in Italian, in plain text. Change it whenever you like:

```
sudo -u emma nano /opt/emma/prompts/system_prompt.txt
sudo systemctl restart emma.service
```

The prompt is read at startup, so **the restart is required**. Some advice: describe behaviour, not identity (“answer in two sentences” works better than “be concise”); state explicitly what she cannot do, so that she does not invent; and keep the file short, because it is sent with every single message and therefore paid for every time.

Two things not to write into it, learnt the hard way. **Never list the tools**: that creates a second inventory which will disagree with the real one — and it already did, in the very sentence forbidding her to recite tools from memory. **Never claim a capability that has no tool behind it**: writing that she can search the web does not make her careful, it makes her confident and wrong.

If you change this file, **commit it**: it is part of the project and is versioned like the code.

## 5.4 What it costs

It depends on the provider. With LLM\_PROVIDER=groq on the free plan it **costs nothing**, within the account’s per-minute request limits: it is the option to prefer if the goal is keeping spending at zero.

With Anthropic you pay per token. Every message consumes input tokens (the system prompt, the conversation window, your question) and output tokens (the answer). With Sonnet and personal use we are talking about a few euros a month, but it depends on how much you write.

The real numbers are in the logs:

```
journalctl -u emma | grep -E "(anthropic|groq) call ok" | tail -20
# ... anthropic call ok (attempt 1): stop_reason=end_turn in=412 out=87
```

in and out are the tokens consumed. The official total is in the Anthropic dashboard; the practical advice is to set a monthly spending limit there.

The parameter that moves the cost most is MAX\_HISTORY\_MESSAGES: doubling it roughly doubles the input tokens of every message, because the whole window is sent again every time.

## 5.5 Known limits

These are declared limits, not defects. Each one has its phase in the roadmap.

- **Few tools that reach the outside world.** She has a clock, facts, an inventory of herself and the development queue — but no weather, calendar, notes, lights or web search. Those are asked for by commissioning them (section 5.6).
- **No voice.** Text only, Telegram only. → v0.5, the Raspberry Pi satellite.
- **Text input only.** Photos, audio and documents sent to the bot are ignored without an answer.
- **Telegram commands do nothing.** /start and the like are ignored: just write normally.
- **One user.** Everyone else is ignored silently, by design.
- **No way to clear the conversation from the phone.** Since v0.2 the memory is persistent, so a restart is no longer enough: to start from scratch, delete the database file and restart the service (section 5.7).

## 5.6 Commissioning development

If EMMA cannot do something, you can ask that she learn to. She does not implement it herself — she records the request, a developer takes it on, and at the end EMMA restarts with the extra capability.

**To file a request** there are two ways:

```
sviluppo: quando ti chiedo il meteo, dimmi che non hai internet
```



With the sviluppo: prefix she files it at once, without discussion. Otherwise you can say it normally and she will offer:

**you:** vorrei che ricordassi i miei appuntamenti **EMMA:** Questa richiede una modifica al mio codice. La registro come lavoro? **you:** sì **EMMA:** Registrata come lavoro #3.

She never files anything on her own initiative: either you are explicit, or she asks.

**To find out how they are going:**

**you:** a che punto sono i lavori? **EMMA:** #3 [implementato e testato, in attesa di essere committato] vorrei che ricordassi i miei appuntamenti — DOMANDA: 53 test verdi. Committo? #4 [in attesa che lo sviluppatore lo prenda in carico] ...

**To answer a question:**

**you:** sì al 3, committa pure **EMMA:** Risposta registrata sul lavoro #3.

Every job stops four times waiting for a yes from you: before implementing, before committing, before publishing to GitHub and before deploying. This is deliberate — the moment when a misunderstood request costs least is before it becomes published code.

**EMMA never writes to you first.** The questions sit there until you ask. If you do not ask for a day, the job stops at the first gate and waits: nothing is lost, but nothing moves on.

**If EMMA says the development session is not active,** like this:

NOTA: l'ultimo contatto con la sessione di sviluppo risale a 2 giorni fa.  
Probabilmente non e' attiva.

...it means exactly that: behind it there is no service that restarts by itself, but a session someone left open on the development PC. If it is closed, requests pile up and nobody collects them. Open it again.

## 5.7 Clearing the memory

The history lives in a SQLite file, by default /opt/emma/data/emma.db, alongside two snapshots. To start from scratch all three have to go:

```
sudo systemctl stop emma.service
sudo -u emma rm -f /opt/emma/data/emma.db /opt/emma/data/emma.db.snapshot*
sudo systemctl start emma.service
```

The database is recreated, empty, on the first message.

**Deleting only emma.db is not enough.** The file would disappear, but the history would remain in the snapshots — and if one day the new database became corrupt, EMMA would restore the conversations you thought you had deleted. The asterisk in the command above is there for exactly this.

If you want to keep the history before deleting it, copy it elsewhere: it is an ordinary SQLite file, readable with sqlite3 emma.db "SELECT \* FROM messages;". Keep it where you keep the backups — it contains your conversations.

# Chapter 6 — Maintenance

## 6.1 Reading the logs

Everything goes to the systemd journal. The commands you will really use:

```
journalctl -u emma -f           # Live (Ctrl+C to quit)
journalctl -u emma -n 100 --no-pager  # the last 100 lines
journalctl -u emma --since "1 hour ago" # the last hour
journalctl -u emma --since today -p err # today's errors only
journalctl -u emma -u emma-backup --since "2 days ago" # service and backup together
```

Every line has the format timestamp | LEVEL | module | message. What the lines you will see most often mean:

| Line                                                         | Meaning                                                                        |
|--------------------------------------------------------------|--------------------------------------------------------------------------------|
| starting emma (provider=..., model=..., history=..., db=...) | startup, with the configuration in use                                         |
| telegram adapter started (long polling)                      | the bot is connected and listening                                             |
| incoming message from chat_id=...                            | a message from you arrived                                                     |
| anthropic call ok (attempt 1): ... in=N out=M                | answer obtained, tokens consumed (with Groq: groq call ok)                     |
| answered chat_id=... (degraded=False)                        | the answer was sent                                                            |
| ignored message from user_id=... (not in whitelist)          | somebody else wrote to the bot                                                 |
| anthropic call failed (attempt 1/3)                          | a failed attempt, it is retrying                                               |
| turn degraded (model_unreachable)                            | every attempt failed, a courteous answer went out                              |
| turn degraded (quota_exhausted)                              | the model quota ran out: the line says for how long                            |
| turn degraded (empty_answer)   (tool_loop_ceiling)           | the other two ways a turn can fall back                                        |
| Groq rate limit is longer than retrying can absorb           | a long limit (usually the daily one): it gives up at once instead of insisting |
| could not read the history, answering without it             | the database is unreadable: it answered anyway, without context                |
| the answer was delivered but not remembered                  | delivered but not stored: next time she will not recall it                     |
| health probe could not read the conversation store           | /health is now answering 503                                                   |
| database integrity check FAILED ...                          | corrupt database: automatic recovery has started (section 6.7)                 |
| RECOVERED: history restored from ...                         | the history was restored from a snapshot                                       |

How much space the journal takes, and how to limit it:

```
journalctl --disk-usage
sudo journalctl --vacuum-time=30d # keeps only the last 30 days
```

## 6.2 The golden rule: back up first

**No code or dependency update without a snapshot taken beforehand.** This is not a recommendation, it is the mandatory first step of every procedure in this chapter. It takes ten seconds and it stands between you and an afternoon of reconstruction.

```
sudo systemctl start emma-backup.service
journalctl -u emma-backup.service -n 20 --no-pager    # check that it went well
ls -lht /mnt/backup/emma/ | head -3                 # the newest archive is from now
```

If the backup fails, **stop**: fix that before touching anything else.

## 6.3 Updating the code: development PC → GitHub → server

The flow is always the same and runs in one direction only. **The code is never edited by hand on the server**: if you did, the next `git pull` would conflict and you would find yourself with two diverging versions and no way to tell which is the good one.

### On the development PC (Windows)

```
# 1. A local snapshot, independent of Git
powershell -ExecutionPolicy Bypass -File .\scripts\backup-dev.ps1
```

```
# 2. The changes, with the checks
ruff format .
ruff check .
pytest
```

```
# 3. Commit and push
git add -A
git commit -m "a description of what changes and why"
git push
```

If the change is a release, update `CHANGELOG.md`, bump the version according to semver and add the tag:

```
git tag -a v0.1.1 -m "v0.1.1 - short description"
git push --tags
```

The semver criterion, in short: **patch** (0.1.x) for fixes that do not change behaviour; **minor** (0.x.0) for new, compatible features; **major** (x.0.0) for changes that break compatibility — in our case, typically an `.env` variable renamed or removed.

Three parts, and no more: a fourth number for “a small change” would only move the question of what counts as small. What answers “exactly which code is running” is the commit, which `/health` and `running_version` already report; the version answers a different question, which is what kind of change it was.

### On the server

```
# 1. MANDATORY BACKUP
sudo systemctl start emma-backup.service
journalctl -u emma-backup.service -n 20 --no-pager

# 2. Note the current version, so you can go back
cd /opt/emma
git rev-parse --short HEAD    # a1b2c3d for instance - write it down

# 3. Fetch the changes
```

```
sudo -u emma git -C /opt/emma pull
```

*# 4. Update the dependencies, if requirements.txt changed*

```
sudo -u emma /opt/emma/.venv/bin/pip install -r /opt/emma/requirements.txt
```

*# 5. Check before restarting*

```
sudo -u emma /opt/emma/.venv/bin/python -m pytest
```

*# 6. Restart*

```
sudo systemctl restart emma.service
```

```
systemctl status emma.service
```

```
curl -s http://127.0.0.1:8000/health
```

If the `git pull` touched files under `systemd/`, copy them again and reload before restarting — `git pull` updates the repository, not the units already installed in `/etc`:

```
sudo cp /opt/emma/systemd/*.service /opt/emma/systemd/*.timer /etc/systemd/system/
```

```
sudo systemctl daemon-reload
```

**Upgrading from a version earlier than 0.2.1.** Two things are needed that were not there before, both once only:

```
sudo apt install -y sqlite3 # for the backup snapshot
```

```
sudo -u emma mkdir -p /opt/emma/data && sudo chmod 700 /opt/emma/data
```

plus the unit copy above, because `emma.service` now declares `ReadWritePaths=/opt/emma/data`: without it, the service cannot write the history.

**The final check: write to the bot from your phone.** A service that is active (running) does not prove the assistant answers; a message does.

If something goes wrong, section 6.6 explains how to go back.

## 6.4 Updating the pinned dependencies

The versions in `requirements.txt` are pinned on purpose. Updating them is a deliberate act, done on the development PC, one package at a time.

*# What has aged*

```
pip list --outdated
```

*# Update one library at a time, not all of them together*

```
pip install --upgrade anthropic
```

```
pip show anthropic | Select-String Version # take the exact number
```

*# write that number into requirements.txt*

*# Check*

```
pytest
```

```
ruff check .
```

```
python main.py # really try it: write to the bot from your phone
```

```
git add requirements.txt
```

```
git commit -m "bump anthropic to X.Y.Z"
```

Why one at a time: if something breaks, you know immediately which library did it. Updating five together, you would spend an hour finding out.

Every so often it is worth regenerating the environment from scratch, to notice a dependency you installed by hand and that is not in `requirements.txt`:

```
Remove-Item -Recurse -Force .venv
python -m venv .venv
.\.venv\Scripts\pip install -r requirements.txt
pytest
```

**Operating-system security updates** — separate and simpler:

```
sudo apt update && sudo apt upgrade -y
sudo systemctl status emma.service      # check that it survived
```

If the upgrade touches the kernel, plan a reboot: `emma.service` is enabled, so it comes back by itself.

## 6.5 Backing up the configuration and the memory

Two files are **not** in Git and have to be protected by the backup: the `.env`, without which the assistant does not start, and the database, which contains all your conversations. The virtualenv is excluded on purpose, because it is rebuilt from `requirements.txt`. It is for these two files that the backup directory has `700` permissions.

The database is not archived by copying it: `backup.sh` takes a snapshot with `VACUUM INTO`, which produces a consistent copy **while the service is writing**, then verifies its integrity and only then includes it. An ordinary copy, taken with `tar` while the service is running, can capture a transaction halfway through: the archive opens without errors and the database inside does not open at all — a fault discovered only on the day of the restore.

Inside the archive the snapshot is called `emma.db` and sits next to `MANIFEST.txt`, not inside `data/` (which is excluded from the tar for exactly this reason). The virtualenv and `~/.cache` are excluded too: the installation directory is also the emma user's home, so pip's cache ends up there, weighing tens of megabytes and entirely rebuildable from `requirements.txt`. A healthy archive weighs a few hundred kilobytes; if you see one of tens of megabytes, something unnecessary got in.

The manifest always states how it went:

```
tar -xzOf /mnt/backup/emma/emma-*.tar.gz MANIFEST.txt | grep database
# database:      emma.db (consistent snapshot, integrity verified)
```

If instead you read `NOT INCLUDED`, that archive contains code and `.env` but not the history: the reason is written on the same line (usually `sqlite3` not installed). **It is not a failed backup** — the rest is valid — but it needs fixing.

If you want a separate copy, for instance before regenerating the API key:

```
sudo cp /opt/emma/.env /mnt/backup/emma/env-$(date +%Y%m%d).bak
sudo chmod 600 /mnt/backup/emma/env-*.bak
```

Do not copy it into Documents, do not email it to yourself, do not put it in an unencrypted cloud service: it is a billable key and control of your bot.

## 6.6 Restoring

**A backup never tested by restoring it is not a backup.** Test it once, today, when you do not need it: finding out that it does not work while you do need it is quite another experience.

### 6.6.1 Going back to an earlier version of the code (without touching the backups)

This is the most frequent case: an update broke something and you want to go back.

```
cd /opt/emma
git log --oneline -10      # the history: the good commit is in there
```

```
sudo -u emma git checkout a1b2c3d # the hash noted at step 2 of section 6.3
sudo systemctl restart emma.service
```

You are now in *detached HEAD*: perfectly fine as a temporary measure. To return to the latest version, `sudo -u emma git checkout main`. If the broken commit is already on GitHub, the clean solution is to fix it on the development PC with a new commit (or a `git revert`) and repeat the cycle in section 6.3 — **do not rewrite history that has already been published**, because that would break the copy on the server.

## 6.6.2 A full restore from an archive

Needed when restoring from Git is not enough: a lost `.env`, a damaged directory, or a new machine.

*# 1. Choose the archive and look at what it contains*

```
ls -lht /mnt/backup/emma/
tar -xzf /mnt/backup/emma/emma-20260829-033012.tar.gz MANIFEST.txt
```

*# 2. Stop the service*

```
sudo systemctl stop emma.service
```

*# 3. Extract into a temporary directory (never straight over the installation)*

```
mkdir -p /tmp/restore
tar -xzf /mnt/backup/emma/emma-20260829-033012.tar.gz -C /tmp/restore
ls /tmp/restore/emma
```

*# 4. Set the current installation aside instead of deleting it*

```
sudo mv /opt/emma /opt/emma.broken-$(date +%Y%m%d)
```

*# 5. Put the restore in place*

```
sudo mv /tmp/restore/emma /opt/emma
sudo chown -R emma:emma /opt/emma
sudo chmod 750 /opt/emma
sudo chmod 600 /opt/emma/.env
```

*# 5b. Put the history back: in the archive the snapshot sits next to the manifest, not inside data/, so it has to be copied by hand. The directory must exist in any case, even with no history to restore: the unit demands it.*

```
sudo -u emma mkdir -p /opt/emma/data
sudo chmod 700 /opt/emma/data
sudo -u emma cp /tmp/restore/emma.db /opt/emma/data/emma.db # if the archive has it
```

*# 6. Recreate the virtualenv: it is not in the archive, by choice*

```
sudo -u emma python3 -m venv /opt/emma/.venv
sudo -u emma /opt/emma/.venv/bin/pip install -r /opt/emma/requirements.txt
```

*# 7. Start again*

```
sudo systemctl start emma.service
systemctl status emma.service
curl -s http://127.0.0.1:8000/health
```

**Check: write to the bot from your phone.** Only then is the restore finished. When you are sure everything works, delete `/opt/emma.broken-*`.

## 6.6.3 Restoring onto a new machine

The same procedure, with chapter 1 in front of it (user, directories, packages, backup disk) and steps 4.6 and 4.7 behind it, to reinstall the systemd units. The contents of the archive give you back the code, the `.env` and the personality:

the rest is the system, and the system is rebuilt from this guide.

### 6.6.4 Restoring from the development PC

The zip files in D:\EmmaBackups contain the project **including the .git directory**, so each one is a complete repository with the full history. If the local repository becomes corrupt:

1. rename the current project folder (do not delete it);
2. extract the most recent zip in its place;
3. `git status` and `git log --oneline -5` to check the history is there;
4. `git push` to bring GitHub back into line, if needed.

## 6.7 Common problems

### The bot does not answer

In this order:

```
systemctl status emma.service           # 1. is the service alive?
journalctl -u emma -n 50 --no-pager     # 2. what do the logs say?
curl -s http://127.0.0.1:8000/health    # 3. is the process answering?
```

- **The service is not active** → look at the error in the logs and go to the relevant case below.
- **The service is active but no incoming message appears in the logs** → the message is not arriving at all. Are you writing to the right bot? Is TELEGRAM\_BOT\_TOKEN the token of *that* bot?
- **The logs show ignored message from user\_id=NNN (not in whitelist)** → this is by far the most common case. That number NNN is your real user ID: put it in TELEGRAM\_ALLOWED\_USER\_ID and restart. (The logs have just told you the answer.)
- **There is incoming message but no answered** → the problem is towards the provider: search for anthropic call failed.

### Messages arrive but EMMA never answers

A treacherous symptom: the service is active, the logs show incoming message, but no answered — and from the phone it is indistinguishable from a bot that is switched off.

```
journalctl -u emma --since "30 min ago" | grep -E "TimedOut|incoming|answered"
```

If you see `telegram.error.TimedOut` with `httpcore.ConnectTimeout` on `connect_tcp`, the process cannot open **new** connections to Telegram, while the long-polling connection — already established — keeps working. That is why messages come in and answers do not go out.

First rule out the machine:

```
sudo -u emma curl -s -o /dev/null -w "%{http_code} in %{time_total}s\n" \
--max-time 15 https://api.telegram.org/
```

If curl answers in a fraction of a second, the network is fine and the problem is the process's connection pool, left hanging after a network disturbance. **It is fixed by a restart**, which recreates it from scratch:

```
sudo systemctl restart emma.service
```

**Careful how you measure whether it is fixed.** `journalctl --since "10 min ago"` can reach back to before the restart and make you count the old errors again, convincing you the problem persists. Use the moment of the restart:

```
R=$(systemctl show emma.service -p ActiveEnterTimestamp --value)
journalctl -u emma --since "$R" | grep -c TimedOut
```

If it happens often, the underlying cause is the network towards Telegram. Measure it:

```
for i in $(seq 1 20); do
    sudo -u emma curl -s -o /dev/null --max-time 6 https://api.telegram.org/ \
    && echo -n . || echo -n X
done; echo
```

On an IPv6-only server a percentage of failures is normal: there is a single reachable address and no IPv4 to fall back to. With the current code a failed send kills the turn silently, so that percentage is also the share of messages that will go unanswered.

### configuration error: required environment variable ... is missing

The .env is missing, is in the wrong place, or the variable is empty.

```
ls -l /opt/emma/.env
sudo grep -c . /opt/emma/.env      # must not be 0
```

The file has to be in the project directory, next to main.py.

### configuration error: TELEGRAM\_ALLOWED\_USER\_ID must be an integer

You put the username in instead of the number. Ask [@userinfobot](#) for your Id and use that, without the @ and without quotes.

### You always get “Non riesco a contattare il cervello”

The Anthropic API is unreachable or is rejecting the key. Look at the logs:

```
journalctl -u emma | grep "anthropic call failed" | tail -5
```

- `AuthenticationError / 401` → the key is wrong, expired or revoked. Create a new one on the console, write it into the .env, `systemctl restart emma`.
- `PermissionDeniedError / 403`, or messages about credit → check the credit and the spending limits on the console.
- `APIConnectionError` → a network or DNS problem on the server:

```
curl -sI https://api.anthropic.com | head -1
resolvectl query api.anthropic.com
```

- `RateLimitError / 429` → **in this case you do not get that sentence**: since 0.3.0 the quota has a message of its own (“*Ho raggiunto il limite di richieste verso il modello*”), with the waiting time when the server states it. EMMA retries short limits by herself — the per-minute ones, which clear in a few seconds — and gives up at once when the required wait is longer than the attempts could cover, because insisting would only make an already-decided refusal slower. On Groq’s free plan the ceiling is **daily** (200,000 tokens): in that case there is nothing to do until the reset.

```
journalctl -u emma | grep "rate limit" | tail -5
```

With `LLM_PROVIDER=groq` the same checks apply, searching for `groq call failed` instead of `anthropic call failed`. A 404 on the model name means `GROQ_MODEL` is not available to your account: list the accessible ones with the command in section 2.8.

### The service keeps restarting

```
journalctl -u emma -n 100 --no-pager | grep -i error
```

Almost always it is a configuration error repeating on every start. After five attempts in five minutes systemd stops by itself: once the problem is fixed, `sudo systemctl reset-failed emma.service && sudo systemctl start emma.service`.



## The backup does not run, or fails

```
journalctl -u emma-backup.service -n 30 --no-pager
systemctl list-timers emma-backup.timer
findmnt /mnt/backup # is the disk mounted?
sudo -u emma df -h /mnt/backup # is there room?
```

- **warning: ... is not on a separate disk** → not an error: the second disk is not mounted and the backup went to /var/backups/emma. If you expected the external disk, `sudo mount -a` and check /etc/fstab (section 1.7.3); the next backup will go back to the right disk on its own.
- **no destination left, no backup taken** → not even the fallback is writable. Check that /var/backups exists and that emma can write to it, and that `ReadWritePaths=` in the unit covers the destination (section 4.7).
- **Permission denied** → either the directory does not belong to emma, or `ReadWritePaths=` is missing from the unit (section 4.7).
- **The timer does not appear in list-timers** → it is not enabled: `sudo systemctl enable --now emma-backup.timer`.

## EMMA remembers nothing, or the service will not start because of the database

Since v0.2 the history lives in `data/emma.db`. Check the file exists and that the emma user can write to it:

```
ls -l /opt/emma/data/
sudo -u emma sqlite3 /opt/emma/data/emma.db "SELECT COUNT(*) FROM messages;"
```

- **Failed to set up mount namespacing** and the service does not start at all → the directory declared in `ReadWritePaths=` does not exist. Create it and restart:  

```
sudo -u emma mkdir -p /opt/emma/data && sudo chmod 700 /opt/emma/data
sudo systemctl restart emma.service
```
- **cannot create the database directory ...** in the logs → the service starts but cannot write. Under `systemd` it is nearly always `ReadWritePaths=` in `emma.service` not covering the path in `MEMORY_DB_PATH`: bring the two into line and reload with `sudo systemctl daemon-reload`.
- **unable to open database file** in the logs → the same permission problem, or `MEMORY_DB_PATH` in the `.env` points at a path that is not writable: `sudo chown -R emma:emma /opt/emma/data`.
- **database is locked** → two EMMA processes are running at once. Check with `systemctl status emma.service` and close the one started by hand.
- **The file is there but the history is empty** → normal after a deletion or on the first start; it fills up again from the next message.

## EMMA restored the memory by herself

If the database becomes corrupt, EMMA notices at startup and repairs herself. She is not silent about it: look in the logs.

```
journalctl -u emma | grep -E "integrity check FAILED|RECOVERED|corrupt"
ls -l /opt/emma/data/
```

What you will see, and what it means:

| Line                                        | Meaning                                                                   |
|---------------------------------------------|---------------------------------------------------------------------------|
| database integrity check FAILED for ...     | the database was damaged                                                  |
| corrupt database kept for inspection at ... | the broken file is there, it was not deleted                              |
| RECOVERED: history restored from ...        | restored from the snapshot; messages written after that snapshot are lost |

| Line                                               | Meaning                                                          |
|----------------------------------------------------|------------------------------------------------------------------|
| snapshot ... is unusable, trying the one before it | the most recent generation was broken, the previous one was used |
| no healthy snapshot available                      | no valid snapshot: EMMA started again with an empty history      |

The damaged file stays at `data/emma.db.corrupt-<date>`. You can try to salvage something from it:

```
sudo -u emma sqlite3 /opt/emma/data/emma.db.corrupt-20260831-143002 \
  ".recover" > /tmp/recovered.sql
```

When you have finished, delete it: it is no use to anyone and takes up space.

**If it happens more than once**, the problem is not SQLite but the disk. Check `dmesg -T | grep -i -E "i/o error|ata"` and the SMART status (`smartctl -a /dev/sda`): a database that becomes corrupt repeatedly is almost always a medium that is dying, and no self-repair makes up for a failing disk.

## The answers are strange or out of character

You edited `prompts/system_prompt.txt` and did not restart: the prompt is read only at startup. `sudo systemctl restart emma.service`.

## git pull reports conflicts

Somebody — probably you — edited files directly on the server. To see what:

```
git -C /opt/emma status
git -C /opt/emma diff
```

If you do not need the local changes, `sudo -u emma git -C /opt/emma checkout -- .` discards them and the pull then goes through. If you do need them, take them back to the development PC and let them come in through the normal flow. And remember the rule: the code is not edited on the server.

## 6.8 A maintenance calendar

| When                   | What                                                                                                                                                                                  |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Automatic, every night | the backup runs at 03:30                                                                                                                                                              |
| Every week             | a glance at <code>journalctl -u emma -p err --since "7 days ago"</code>                                                                                                               |
| Every month            | <code>sudo apt update &amp;&amp; sudo apt upgrade</code> ; check the spending on the Anthropic console; <code>ls -lh /mnt/backup/emma/</code> to verify the archives really are there |
| Every three months     | <code>pip list --outdated</code> on the development PC and a considered update; <b>a restore test</b> (section 6.6)                                                                   |
| Once, now              | the first restore test, before you need it                                                                                                                                            |

# Appendix A — Reference commands

## *# Service*

```
sudo systemctl status emma.service
sudo systemctl restart emma.service
sudo systemctl stop emma.service
sudo systemctl start emma.service
```

## *# Logs*

```
journalctl -u emma -f
journalctl -u emma -n 100 --no-pager
journalctl -u emma --since today -p err
```

## *# Health*

```
curl -s http://127.0.0.1:8000/health
```

## *# Backup*

```
sudo systemctl start emma-backup.service
systemctl list-timers emma-backup.timer
ls -lht /mnt/backup/emma/ | head
```

## *# Memory: clear the history (snapshots included)*

```
sudo systemctl stop emma.service
sudo -u emma rm -f /opt/emma/data/emma.db /opt/emma/data/emma.db.snapshot*
sudo systemctl start emma.service
```

## *# Memory: how the last automatic recovery went*

```
journalctl -u emma | grep -E "integrity check FAILED|RECOVERED"
```

## *# Updating (after the backup)*

```
sudo -u emma git -C /opt/emma pull
sudo -u emma /opt/emma/.venv/bin/pip install -r /opt/emma/requirements.txt
sudo -u emma /opt/emma/.venv/bin/python -m pytest
sudo systemctl restart emma.service
```

## *# History, and going back*

```
git -C /opt/emma log --oneline -10
sudo -u emma git -C /opt/emma checkout <hash>
sudo -u emma git -C /opt/emma checkout main
```

## Appendix B — The .env variables

| Variable                 | Mandatory             | Default                   | Notes                                       |
|--------------------------|-----------------------|---------------------------|---------------------------------------------|
| LLM_PROVIDER             | no                    | anthropic                 | anthropic or groq                           |
| ANTHROPIC_API_KEY        | if provider=anthropic | —                         | starts with sk-ant-; it is a secret         |
| ANTHROPIC_MODEL          | no                    | claude-sonnet-4-6         | any valid identifier                        |
| GROQ_API_KEY             | if provider=groq      | —                         | starts with gsk_; it is a secret            |
| GROQ_MODEL               | no                    | openai/gpt-oss-120b       | depends on the Groq account plan            |
| TELEGRAM_BOT_TOKEN       | yes                   | —                         | from @BotFather; it is a secret             |
| TELEGRAM_ALLOWED_USER_ID | yes                   | —                         | a number, not a username; from @userinfobot |
| MAX_HISTORY_MESSAGES     | no                    | 20                        | messages in the window; affects the cost    |
| MEMORY_DB_PATH           | no                    | data/emma.db              | the SQLite file; created automatically      |
| SYSTEM_PROMPT_PATH       | no                    | prompts/system_prompt.txt | relative to the project directory           |
| BACKUP_DIR               | no                    | /mnt/backup/emma          | read by backup.sh                           |
| BACKUP_KEEP              | no                    | 14                        | archives kept by the rotation               |

**Note:** the SQLite file (data/emma.db) and its two snapshots hold the conversation history, the facts, the job queue and the tool state, and must never end up in a Git commit — .gitignore already excludes the whole data/ directory. To clear the memory the snapshots have to be deleted too, otherwise an automatic recovery could bring them back: see section 5.7.

## Appendix C — Where to look when something does not add up

| Question                              | Answer                                                                                                              |
|---------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Is the service alive?                 | <code>systemctl status emma.service</code>                                                                          |
| What happened?                        | <code>journalctl -u emma -n 100 --no-pager</code>                                                                   |
| Is the process answering?             | <code>curl -s http://127.0.0.1:8000/health</code>                                                                   |
| Which version is running?             | <code>curl -s http://127.0.0.1:8000/health</code> , or ask her                                                      |
| When was the last backup?             | <code>ls -lht /mnt/backup/emma/ \   head -3</code>                                                                  |
| How much am I spending?               | <code>journalctl -u emma \   grep -E "(anthropic\ groq) call ok" \   tail -20</code>                                |
| Which settings are active?            | the startup log: <code>journalctl -u emma \   grep "starting emma"</code>                                           |
| How many conversations are in memory? | <code>sudo -u emma sqlite3 /opt/emma/data/emma.db "SELECT conv_id, COUNT(*) FROM messages GROUP BY conv_id;"</code> |
| Why was it decided this way?          | REVISIONONE.md, and chapter 2 of this guide                                                                         |

---

*EMMA v0.4.0 — guide updated on 3 September 2026. The source of this document is docs/GUIDA.md: edit it there and regenerate the PDF, so that the two versions do not diverge.*